Advanced Divide and Conquer Algorithms

Strassen Matrix Multiplication, Karatsuba Multiplication, Closest Pair of Points / Median of Medians

---

1. Strassen Matrix Multiplication

Question 1: Graphics Engine Transform Pipeline

A 3D graphics engine repeatedly multiplies large transformation matrices (rotation, scaling, projection) chained together every frame. For large matrix sizes, reducing the number of scalar multiplications directly improves frame rate. Implement Strassen's algorithm, which needs only 7 multiplications per 2×2 block instead of 8, and verify it produces the same result as standard multiplication.

Approach:

Strassen's algorithm recursively splits each n×n matrix into four (n/2)×(n/2) submatrices, computes 7 cleverly combined products (M1–M7) instead of 8, and recombines them into the four quadrants of the result. The base case (1×1) is a single scalar multiplication. Matrices whose size is not a power of two are padded with zeros first.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n^2.807)	7 recursive multiplications of half-size matrices instead of 8, giving n^log2(7) ≈ n^2.807 vs. n^3 for standard multiplication.
Space	O(n²)	Extra submatrices (M1–M7, A_ij, B_ij) are allocated at each recursion level.

Test Cases:

```
def standard_multiply(A, B):
    n, m, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(n)]
    for i in range(n):
        for j in range(p):
            for k in range(m):
                C[i][j] += A[i][k] * B[k][j]
    return C

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
assert strassen_multiply(A, B) == standard_multiply(A, B)

A3 = [[1,2,3],[4,5,6],[7,8,9]]  # non-power-of-2 size, tests padding
B3 = [[9,8,7],[6,5,4],[3,2,1]]
assert strassen_multiply(A3, B3) == standard_multiply(A3, B3)
print('All test cases passed!')
```

Question 2: Comparing Multiplication Counts

Your team is deciding whether Strassen's algorithm is worth the added code complexity for a scientific computing library. Write a function that counts the theoretical number of scalar multiplications used by Strassen's algorithm versus the standard algorithm for an n×n matrix, and use it to justify the crossover point where Strassen becomes worthwhile.

Approach:

The standard algorithm performs n^3 multiplications. Strassen's algorithm performs 7 multiplications per level of recursion instead of 8, so for a matrix padded to size 2^k, the total is 7^k. Compare the two counts across a range of sizes to see where Strassen starts winning.

Complexity Analysis:

Aspect	Complexity	Explanation
Strassen	O(n^2.807)	7^k multiplications where 2^k is the padded matrix size.
Standard	$O(n^3)$	Direct triple-nested-loop multiplication.

Test Cases:

```
assert count_multiplications_strassen(2) == 7
assert count_multiplications_standard(2) == 8
assert count_multiplications_strassen(4) == 49  # 7^2
assert count_multiplications_standard(4) == 64  # 4^3
assert count_multiplications_strassen(64) < count_multiplications_standard(64)
print('All test cases passed!')
```

Question 3: Non-Square / Padded Matrices in a Simulation

A physics simulation multiplies matrices whose sizes are not always powers of two (e.g. a 3×3 stress tensor or a 5×5 grid transform). Extend Strassen's algorithm to handle arbitrary matrix sizes by padding with zeros before recursing, and trimming the result back down afterward.

Approach:

Pad both input matrices with zero rows/columns up to the next power of two, run the standard recursive Strassen algorithm on the padded matrices, then slice the result back down to the original output dimensions (rows of A by columns of B).

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n^2.807)	Padding adds at most a constant factor since the padded size is less than double the original size.
Space	$O(n^2)$	Padded matrices and intermediate submatrices require additional memory.

Test Cases:

```
def standard_multiply(A, B):
    n, m, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(n)]
    for i in range(n):
        for j in range(p):
            for k in range(m):
                C[i][j] += A[i][k] * B[k][j]
```

```
    return C

A5 = [[i + j for j in range(5)] for i in range(5)]
B5 = [[1 if i == j else 0 for j in range(5)] for i in range(5)]
assert strassen_arbitrary_size(A5, B5) == standard_multiply(A5, B5)  # identity check

A_rect = [[1, 2, 3], [4, 5, 6]]      # 2x3
B_rect = [[7, 8], [9, 10], [11, 12]]  # 3x2
assert strassen_arbitrary_size(A_rect, B_rect) == standard_multiply(A_rect, B_rect)
print('All test cases passed!')
```

Question 4: Benchmarking Strassen vs. Standard Multiplication

A machine learning framework multiplies large weight matrices during training. Benchmark Strassen's algorithm against standard triple-loop multiplication on random matrices of increasing size to measure the practical speedup (or overhead) in Python.

Approach:

Generate random square matrices of several sizes, time both the standard and Strassen implementations on each, and compare. Note that Strassen's constant-factor overhead (extra additions and recursive calls) means it often only pays off for large n in a pure-Python implementation, even though its asymptotic complexity is lower.

Complexity Analysis:

Aspect	Complexity	Explanation
Standard	$O(n^3)$	Simple triple loop; low constant overhead per operation.
Strassen	O(n^2.807)	Lower asymptotic growth, but higher constant factor from extra matrix additions in pure Python.

Test Cases:

```
A, B = random_matrix(8), random_matrix(8)
assert strassen_multiply(A, B) == standard_multiply(A, B)  # correctness before timing
std_t, strassen_t = benchmark(16)
assert std_t >= 0 and strassen_t >= 0
print('All test cases passed!')
```

Question 5: Recursive Depth and Base Case Tuning

For small matrices, Strassen's recursive overhead outweighs its multiplication savings. Modify Strassen's algorithm to fall back to standard multiplication once the submatrix size drops below a chosen threshold, and test that the hybrid version still produces correct results.

Approach:

Add a threshold parameter. During recursion, once the current submatrix size is at or below the threshold, switch to the direct triple-loop multiplication instead of continuing to split, avoiding unnecessary recursive overhead for small blocks.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n^2.807) amortized	Threshold avoids recursion overhead for small blocks while

Aspect	Complexity	Explanation
		keeping Strassen's asymptotic benefit for large ones.
Tuning	Threshold-dependent	Optimal threshold depends on the constant-factor cost of additions vs. multiplications on the target hardware.

Test Cases:

```
A = [[random.randint(-5,5) for _ in range(4)] for _ in range(4)]
B = [[random.randint(-5,5) for _ in range(4)] for _ in range(4)]
assert strassen_hybrid(A, B, threshold=2) == standard_multiply(A, B)
assert strassen_hybrid(A, B, threshold=4) == standard_multiply(A, B)  # threshold == n falls back immediately
print('All test cases passed!')
```

2. Karatsuba Multiplication

Question 1: Cryptographic Key Generation

An RSA key-generation library must multiply very large integers (hundreds of digits) far faster than schoolbook multiplication allows. Implement Karatsuba's algorithm, which reduces the four sub-multiplications of schoolbook multiplication to three, and verify it matches Python's built-in multiplication.

Approach:

Split each number into a high and low half at the middle digit. Instead of computing all four cross products (high1×high2, high1×low2, low1×high2, low1×low2), compute only three: z2 = high1×high2, z0 = low1×low2, and z1 = (high1+low1)×(high2+low2) − z2 − z0. Combine these with positional shifts to get the final product.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n^1.585)	Three recursive multiplications of half-size numbers, n^log2(3), instead of four for schoolbook multiplication.
Space	O(n)	Recursion depth is O(log n); each level allocates a constant number of new integers.

Test Cases:

```
assert karatsuba(1234, 5678) == 1234 * 5678
assert karatsuba(123456789, 987654321) == 123456789 * 987654321
assert karatsuba(9, 9) == 81       # base case
assert karatsuba(0, 12345) == 0    # zero operand
big1, big2 = int('9'*50), int('8'*50)
assert karatsuba(big1, big2) == big1 * big2
print('All test cases passed!')
```

Question 2: Arbitrary-Precision Arithmetic Library

A financial system needs an arbitrary-precision arithmetic library that multiplies numbers far larger than a machine word without losing precision. Compare Karatsuba's algorithm against naive schoolbook multiplication as digit counts grow, to justify which to ship in the library.

Approach:

Implement both a naive $O(n^2)$ schoolbook multiplication (using string/digit manipulation to simulate a low-level implementation) and Karatsuba, then compare their results and relative operation counts as the numbers grow larger.

Complexity Analysis:

Aspect	Complexity	Explanation
Schoolbook	$O(n^2)$	Each digit of x is multiplied against the entirety of y.
Karatsuba	O(n^1.585)	Recursive splitting reduces the number of digit-level multiplications needed as n grows.

Test Cases:

```
for digits in [2, 4, 8, 16, 32]:
    x = int('7' * digits)
    y = int('3' * digits)
    assert karatsuba(x, y) == x * y
print('All test cases passed!')
```

Question 3: Counting Recursive Calls

Your team wants to understand how Karatsuba's divide-and-conquer structure behaves in practice. Instrument the algorithm to count the number of recursive multiplication calls it makes for a given input, and compare that against the $O(n^2)$ call count schoolbook multiplication would need.

Approach:

Add a mutable counter (e.g. a single-element list) that increments once per call to the Karatsuba function, including base cases. Run it on numbers of increasing digit length and observe how the call count grows sub-quadratically.

Complexity Analysis:

Aspect	Complexity	Explanation
Recursive Calls	O(n^1.585)	Each call spawns 3 further calls on roughly half-size operands, following $T(n) = 3T(n/2) + O(n)$.
Schoolbook (for comparison)	$O(n^2)$ digit multiplications	Every digit pair must be multiplied directly, with no recursive structure to exploit.

Test Cases:

```
counter = [0]
result = karatsuba_with_count(1234, 5678, counter)
assert result == 1234 * 5678
assert counter[0] > 0

counter2 = [0]
karatsuba_with_count(9, 9, counter2)
assert counter2[0] == 1  # base case makes exactly one call
print('All test cases passed!')
```

Question 4: Recursive Split Visualizer

You are building an educational tool showing how Karatsuba splits a multiplication problem into subproblems. Implement a version that records each split (the high/low halves at every recursion level) so it can be rendered as a recursion tree.

Approach:

At each recursive call, before returning, log the operands, their high/low splits, and the depth of recursion into a shared list. This trace can then be used to draw a tree diagram showing how the original multiplication decomposes into smaller ones.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n^1.585)	Same recursive structure as standard Karatsuba; tracing adds only O(1) work per call.
Space	O(n^1.585)	The trace list grows proportionally to the total number of recursive calls made.

Test Cases:

```
result, trace = karatsuba_traced(1234, 56)
assert result == 1234 * 56
assert len(trace) > 0
assert trace[0][0] == 0  # first entry is at recursion depth 0
print('All test cases passed!')
```

Question 5: Polynomial Multiplication

A computer algebra system needs to multiply two polynomials represented as coefficient lists. Adapt the divide-and-conquer idea behind Karatsuba to multiply polynomials with fewer coefficient-multiplications than the naive $O(n^2)$ convolution.

Approach:

Karatsuba's trick generalizes directly to polynomials: split each polynomial into a high-degree and low-degree half, compute three products of half-size polynomials (instead of four), and combine them with the appropriate degree shifts, mirroring the integer version exactly.

Complexity Analysis:

Aspect	Complexity	Explanation
Naive Convolution	$O(n^2)$	Every coefficient of p1 is multiplied against every coefficient of p2.
Karatsuba-style	O(n^1.585)	Three recursive half-size polynomial multiplications instead of four, mirroring the integer case.

Test Cases:

```
assert multiply_polynomials_naive([1, 2], [3, 4]) == [3, 10, 8]  # (1+2x)(3+4x)

p1, p2 = [1, 2, 3, 4], [5, 6, 7, 8]
naive_result = multiply_polynomials_naive(p1, p2)
karatsuba_result = karatsuba_poly(p1, p2)[:len(naive_result)]
assert karatsuba_result == naive_result
print('All test cases passed!')
```

3. Closest Pair of Points / Median of Medians

Question 1: Closest Pair of Cities on a Map

A GIS application needs to find the two closest cities out of thousands of coordinates to flag a possible data-entry duplicate. Checking every pair would be $O(n^2)$; implement the divide-and-conquer closest-pair algorithm to solve it in O(n log n).

Approach:

Sort points by x-coordinate, recursively find the closest pair in the left and right halves, take the smaller of the two distances d, then check a narrow vertical strip of width 2d around the dividing line (points sorted by y) since only those points can possibly form a closer pair across the boundary.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n log n)	Initial sort is O(n log n); the recurrence T(n) = 2T(n/2) + O(n) for the strip check also resolves to O(n log n).
Space	O(n)	Sorted-by-y sublists are recreated at each level of recursion.

Test Cases:

```
pts = [(2,3),(12,30),(40,50),(5,1),(12,10),(3,4)]
pair, d = closest_pair_of_points(pts)
brute_pair, brute_d = brute_force_closest(pts)
assert abs(d - brute_d) < 1e-9

import random
random.seed(0)
random_pts = [(random.uniform(0,100), random.uniform(0,100)) for _ in range(50)]
_, d2 = closest_pair_of_points(random_pts)
_, brute_d2 = brute_force_closest(random_pts)
assert abs(d2 - brute_d2) < 1e-9
print('All test cases passed!')
```

Question 2: Collision Detection in a Game Engine

A 2D game engine needs to detect the two nearest objects on screen every frame to check for potential collisions among hundreds of moving sprites. Use the closest-pair divide-and-conquer algorithm each frame instead of an $O(n^2)$ pairwise scan.

Approach:

Treat each sprite's position as a point. Re-run the closest-pair algorithm on the current frame's positions; if the minimum distance found is below a collision threshold, flag those two sprites for collision handling.

Complexity Analysis:

Aspect	Complexity	Explanation
Time per frame	O(n log n)	Each frame re-runs the O(n log n) closest-pair algorithm rather than an $O(n^2)$ pairwise check.
Trade-off	Amortized cost	For very small n (a handful of sprites) the constant overhead of divide-and-conquer may not beat brute force, but it scales far better as sprite count grows.

Test Cases:

```
sprites = [(0,0), (1,1), (50,50), (100,100), (1.2,0.9)]
pair, min_dist = detect_potential_collision(sprites, threshold=2.0)
assert pair is not None and min_dist <= 2.0

far_sprites = [(0,0), (100,100), (200,200)]
pair2, min_dist2 = detect_potential_collision(far_sprites, threshold=1.0)
assert pair2 is None  # nothing within threshold
print('All test cases passed!')
```

Question 3: Sensor Node Placement

A wireless sensor network wants to check whether any two of its hundreds of deployed sensor nodes are placed too close together (risking signal interference). Use the closest-pair algorithm to efficiently find the minimum inter-node distance across the whole network.

Approach:

Run the closest-pair divide-and-conquer algorithm once on all node coordinates to obtain the minimum distance in O(n log n) time, then compare it against the network's minimum-safe-distance requirement.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n log n)	A single closest-pair computation is enough to validate spacing across the whole network.
Space	O(n)	Sorted coordinate lists are maintained through the recursion.

Test Cases:

```
nodes = [(0,0), (10,10), (10.5,10.2), (30,40), (31,41)]
ok, pair, min_dist = check_minimum_spacing(nodes, min_safe_distance=1.0)
assert ok == False  # (10,10) and (10.5,10.2) are too close

spaced_nodes = [(0,0), (10,10), (20,20), (30,30)]
ok2, _, _ = check_minimum_spacing(spaced_nodes, min_safe_distance=1.0)
assert ok2 == True
print('All test cases passed!')
```

Question 4: Guaranteed-Time Median Salary Lookup

An HR analytics dashboard must report the median salary from a very large employee dataset, and it needs a worst-case time guarantee (unlike Quickselect, whose worst case is $O(n^2)$ with a bad pivot). Implement the median-of-medians selection algorithm, which guarantees O(n) worst-case time.

Approach:

Split the array into groups of 5, find the median of each small group (cheaply, by sorting 5 elements), then recursively find the median of those group-medians to use as a pivot. This pivot is guaranteed to eliminate a constant fraction of elements on each partition step, giving a worst-case linear-time selection algorithm.

Complexity Analysis:

Aspect	Complexity	Explanation
Time	O(n) worst case	Median-of-medians pivot guarantees at least 30% of elements are eliminated at every partition step, so T(n) = T(n/5) + T(7n/10) + O(n) resolves to O(n).
Space	O(n)	New lists (low/high/equal, and group medians) are created at each recursive call.

Test Cases:

```
data = [12, 3, 5, 7, 4, 19, 26]
for k in range(len(data)):
    assert median_of_medians_select(data, k) == sorted(data)[k]

import random
random.seed(1)
big_data = [random.randint(0, 10000) for _ in range(500)]
sorted_big = sorted(big_data)
for k in [0, 10, 250, 499]:
    assert median_of_medians_select(big_data, k) == sorted_big[k]
print('All test cases passed!')
```

Question 5: Kth Fastest Delivery Time (Worst-Case Guarantee)

A logistics company wants to find the Kth fastest delivery time out of millions of trip records for SLA reporting, and needs a hard guarantee that this lookup never degrades to $O(n^2)$, which ordinary Quickselect can do on adversarial or sorted input. Use median-of-medians selection to guarantee O(n) worst-case performance.

Approach:

Reuse the median-of-medians selection routine to directly find the element of a given rank (the Kth smallest delivery time) without fully sorting the dataset, and confirm its worst-case behavior stays linear even on already-sorted input, which is the classic worst case for naive Quickselect.

Complexity Analysis:

Aspect	Complexity	Explanation
Time (any input order)	O(n) worst case	Unlike naive Quickselect, median-of-medians pivot selection guarantees linear time even on already-sorted or adversarial input.
Naive Quickselect (for comparison)	$O(n^2)$ worst case	A poorly chosen pivot (e.g. always picking the first element on sorted input) can degrade performance to quadratic time.

Test Cases:

```
already_sorted = list(range(1, 501))  # classic Quickselect worst case
assert kth_smallest_delivery_time(already_sorted, 0) == 1
assert kth_smallest_delivery_time(already_sorted, 499) == 500
assert kth_smallest_delivery_time(already_sorted, 250) == 251

delivery_times = [45,30,60,25,50,40,35,55,20,65]
assert kth_smallest_delivery_time(delivery_times, 0) == min(delivery_times)
print('All test cases passed!')
```